LLMs at the Edge: Performance and Efficiency Evaluation with Ollama on Diverse Hardware

Donghao HUANG[1,2], Zhaoxia WANG[1]
School of Computing and Information Systems, Singapore Management University, Singapore[1]
Research and Development, Mastercard, Singapore[2]
dh.huang.2023@smu.edu.sg; zxwang@smu.edu.sg

***Abstract*—Energy efficiency is a critical consideration in deploying large language models (LLMs) due to their significant environmental footprint. This study addresses this challenge by leveraging Ollama, an open-source framework that facilitates efficient local deployment of LLMs, thereby reducing energy consumption alongside enhancing privacy and accessibility. We evaluate the performance and efficiency of open-source LLMs from the Qwen2.5 and Llama3 families (0.5B–90B parameters) across diverse hardware platforms, including consumer-grade devices. Our evaluation spans high-end systems (RTX A6000, RTX 4090), consumer laptops like Apple M3/M4, and accessible platforms like NVIDIA Jetson AGX Orin, showcasing pathways to widespread AI adoption. Key findings include: (1) Ollama's quantization enables efficient LLM deployment on consumer hardware while preserving performance; (2) Windows Subsystem for Linux enhances speeds across all models, delivering up to 21× improvements for the smallest model while also improving energy efficiency, thereby broadening adoption on common computing platforms; (3) Medium-sized models (7B–32B) achieve an ideal balance of performance and efficiency; (4) Larger models achieve the highest performance and exhibit better cross-platform consistency but are significantly less energy efficient. These results highlight strategies for inclusive, privacy-preserving, and sustainable AI deployment, advancing the democratization of LLM technology while reducing energy consumption and infrastructure demands.**

***Index Terms*—Large Language Models (LLMs), edge AI, Ollama framework, AI democratization, privacy preservation, sustainable computing, inclusive AI, hardware performance benchmarking, model quantization**

I. Introduction

Large Language Models (LLMs) have revolutionized natural language processing, enabling diverse applications such as text summarization, machine translation, and complex question answering (1). However, deploying these sophisticated models on resource-constrained edge devices remains challenging due to limitations in computational power, memory, and energy efficiency (2; 1). Addressing these challenges requires innovative deployment strategies that balance model performance with resource constraints, particularly for edge AI environments.

The democratization of LLMs through edge deployment holds significant potential for social good. By enabling local execution, edge LLMs enhance privacy in sensitive domains like healthcare and education, reduce the environmental footprint of AI deployment, and expand access to AI in regions with limited cloud infrastructure. Moreover, edge deployment lowers economic barriers, making AI technologies accessible to resource-constrained organizations and communities (3).

The growing demand for edge-based LLMs is driven by their advantages, including reduced latency, enhanced privacy, and offline operational capabilities. These features make edge LLMs indispensable for critical infrastructure and real-time applications (1). Optimization techniques such as quantization (4; 5) have emerged to enable efficient deployment across heterogeneous edge devices, from smartphones and wearables to IoT sensors and embedded systems (1).

In this context, Ollama emerges as a lightweight, open-source framework designed to simplify local LLM deployment and execution[1]. By offering an intuitive API for model management, Ollama addresses deployment complexities while enhancing transparency, privacy, and performance. Its cross-platform compatibility across macOS, Linux, and Windows allows organizations to leverage existing hardware for AI applications efficiently.

This paper evaluates open-source LLMs for edge AI applications using Ollama, focusing on models from the *Qwen2.5* (6) and *Llama3* (7) families. The evaluation covers a range of model sizes (0.5B–90B parameters) and precisions (4-bit quantization and FP16) across diverse hardware platforms.

Our main contributions are as follows:

- Leveraged Ollama to assess the performance and efficiency of Qwen2.5 and Llama3 models across six hardware platforms, including Apple M3/M4 laptops, NVIDIA GPUs (RTX A6000 for desktop and RTX 4090 for both desktop and laptop configurations), and edge devices such as the NVIDIA Jetson AGX Orin, providing valuable insights into performance trade-offs for edge AI applications.
- Demonstrated that running Ollama on the Windows Subsystem for Linux (WSL) achieves up to 21× higher inference speeds for smaller models compared to native Windows, underscoring WSL's critical role in maximizing efficiency.
- Established that Ollama's `Q4_K_M` quantization outperforms the 4-bit implementation of bitsandbytes, delivering comparable or superior performance to FP16 while requiring less memory and enabling faster inference speeds. Medium-sized models (7B–32B) achieve an optimal balance between performance and efficiency, with

[1]https://ollama.com/

`qwen2.5:32B` attaining a 96.75% F1 score, practical inference speeds, and moderate energy consumption.

- Developed a comprehensive evaluation framework that measures F1 score, throughput, memory usage, latency, and energy consumption across edge platforms. The publicly available codebase[2] supports reproducibility and facilitates future research.

II. Related Work

The deployment of LLMs on edge devices has garnered increasing attention, driven by the rising demand for localized, low-latency, and privacy-preserving AI capabilities. Existing literature highlights several advancements and challenges in the efficient deployment and inference of LLMs on resource-constrained edge hardware.

Significant progress has been made in optimizing LLMs for edge deployment using techniques such as quantization, pruning, and knowledge distillation. Quantization methods, including INT4 and FP8 precision, have shown promise in reducing model size and computational demands while maintaining performance (1; 2; 8). Pruning techniques—both structured and unstructured—streamline models by removing redundant parameters, enhancing their feasibility for edge applications (1). Additionally, knowledge distillation facilitates the transfer of capabilities from large models to smaller, edge-compatible versions, preserving task-specific performance (1). The application of these techniques to transformer-based architectures remains an active research area, as their high-dimensional operations pose unique challenges (1; 2).

Despite advancements, several challenges persist in deploying LLMs on edge devices. Limited memory and computational resources on devices like Raspberry Pi and Jetson AGX Orin constrain the feasible model sizes and require aggressive compression strategies (2; 8). Trade-offs between model precision and accuracy, as well as the latency and energy costs associated with different hardware platforms, necessitate careful consideration in model and platform selection (9; 8). Additionally, ensuring compatibility across heterogeneous edge environments remains a key obstacle (1).

This research presents LLMs for Edge AI, extending the existing body of work by evaluating a diverse range of open-source LLMs using the Ollama framework across multiple edge platforms.

III. LLMs for Edge AI

A. Dataset and Prompt Engineering for LLMs at the Edge Evaluation

This study utilizes the *Global Maritime Risk Incident Dataset (GMRID) v2* introduced in (10), extending it into *GMRID v3*. The extended dataset includes a new label column, *gpt-4o_label*, generated using OpenAI's GPT-4o model (List 2), and a composite *Headline_Details* column created by merging the *Headline* and *Details* fields, which serves as input for LLM prompts.

[2]https://github.com/inflaton/llms-at-edge

GMRID v3 was chosen for its domain specificity to the maritime industry, where LLM applications are relatively unexplored, ensuring the reliability of this evaluation.

Building on the few-shot techniques from (10), we developed a prompting strategy tailored to the *GMRID v3* dataset. The system prompt template is shown in Listing 1.

Listing 1. System Prompt for LLM Classification

```
Task: Classify Inputs into Predefined Categories
Your primary objective is to analyze the given input and
assign it to one of the predefined categories: {
categories_list}. Evaluate the content carefully and use
the defining characteristics of each category to ensure an
accurate classification.
[Additional details omitted for clarity]
Instructions for Output:

1. Once the category is identified, provide "specific tags"
 by selecting from the list corresponding to the identified
 category, as defined in the JSON.
2. Ensure the selected "specific tags" accurately reflect
the details and context of the input.
Output Format:

Return your classification in the following JSON format:
{
  "category": "<Selected Category>",
  "specific_tags": ["<Selected Tag 1>", ...]
}
```

This system prompt template extends the one in (10) by:

- Adding instructions to output specific tags.
- Mandating JSON as the output format.

Correspondingly, exemplars are retrieved from the *GMRID v3* training set and formatted as shown in Listing 2.

Listing 2. Exemplars Used in Few-Shot Prompting

```
Example Inputs and Outputs:
- Input:
[Additional details omitted for clarity]
- Output:
{
  "category": "Weather",
  "specific_tags": ["Severe Winds"]
}
```

To improve few-shot learning, we ensure each sample belongs to a different category until the shot number exceeds the total number of categories, which is eight. Additionally, we introduced more granular shot numbers: 0, 1, 2, 4, 8, and 10, replacing the original sequence of 0, 1, 3, 5, and 10.

B. Evaluation Metrics

Model performance was evaluated using predictive accuracy on the held-out test data, calculated with the weighted F1 score based on LLM outputs and the *gpt-4o_label* column.

To assess inference speed, the following metrics were used:

1) **Mean Evaluation Time**: The average time (in seconds) required for the LLM to complete a classification task. It is abbreviated as **Time (s)** and is calculated as:

$$\text{Mean Evaluation Time} = \frac{\text{Total Evaluation Time (s)}}{\text{Size of Test Dataset}}$$

2) **Throughput**: This metric indicates the number of tokens processed per second. It is abbreviated as **T-put (t/s)** and calculated as:

$$\text{Throughput} = \frac{\text{Total Input \& Output Tokens (t)}}{\text{Total Evaluation Time (s)}}$$

To evaluate energy efficiency, we adopted the following metrics from (11):

1) **Energy per Inference**: This metric quantifies the energy required to process a single input sequence, measured in Joules (J). It is calculated as:

$$\text{Energy per Inference (J)} = \text{Time (s)} \times P_{\text{CPU+GPU}}(W)$$

where $P_{\text{CPU+GPU}}$ is the average power consumption of CPU and GPU during inference, excluding idle power.

2) **Performance-to-Power Ratio**: The performance achieved per unit of energy consumed:

$$\text{Performance-to-Power Ratio} = \frac{\text{F1 Score (\%)}}{\text{Energy Consumed (J)}}$$

These metrics provide a holistic evaluation of predictive accuracy, inference speed, and energy efficiency, ensuring practical insights into model deployment on various hardware configurations.

C. Model Selection and Specifications

Table I details the evaluated LLMs, focusing on the Qwen2.5 and Llama3 families. Models with up to 14 billion parameters are evaluated in both quantized (`Q4_K_M`) and FP16 versions, while larger models are assessed only in quantized form due to memory constraints.

Ollama's standardized naming conventions ensure consistency and reproducibility. This diverse range of architectures and scales enables a comprehensive analysis of performance and feasibility for edge AI applications.

Name	Size	Precision
qwen2.5:0.5b	397 MB	Q4_K_M
qwen2.5:0.5b-instruct-fp16	994 MB	FP16
llama3.2:1b	1.3 GB	Q4_K_M
llama3.2:1b-instruct-fp16	2.5 GB	FP16
qwen2.5:1.5b	986 MB	Q4_K_M
qwen2.5:1.5b-instruct-fp16	3.1 GB	FP16
llama3.2:3b	2.0 GB	Q4_K_M
llama3.2:3b-instruct-fp16	6.4 GB	FP16
qwen2.5:3b	1.9 GB	Q4_K_M
qwen2.5:3b-instruct-fp16	6.2 GB	FP16
qwen2.5:7b	4.7 GB	Q4_K_M
qwen2.5:7b-instruct-fp16	15 GB	FP16
llama3.1:8b	4.9 GB	Q4_K_M
llama3.1:8b-instruct-fp16	16 GB	FP16
llama3.2-vision:11b	7.9 GB	Q4_K_M
llama3.2-vision:11b-instruct-fp16	21 GB	FP16
qwen2.5:14b	9.0 GB	Q4_K_M
qwen2.5:14b-instruct-fp16	29 GB	FP16
qwen2.5:32b	19 GB	Q4_K_M
llama3.1:70b	42 GB	Q4_K_M
llama3.3:70b	42 GB	Q4_K_M
qwen2.5:72b	47 GB	Q4_K_M
llama3.2-vision:90b	54 GB	Q4_K_M

TABLE I
SPECIFICATIONS OF THE EVALUATED QWEN2.5 AND LLAMA3 OLLAMA MODELS. THE SIZE VALUES REPRESENT THE FILE SIZES OF THE MODEL WEIGHTS. THE PRECISION VALUES `Q4_K_M` AND FP16 CORRESPOND TO QUANTIZED 4-BIT PRECISION AND HALF-PRECISION FLOATING-POINT FORMATS, RESPECTIVELY.

D. Hardware Infrastructure and Test Environments

To evaluate the selected Ollama models, we conducted performance testing across six hardware systems and eight configurations, covering diverse computing environments:

1) **Apple M4 Max MacBook Pro:** Running macOS Sequoia 15.1, this system features the M4 Max chip with 48 GB of unified memory, optimized for high-performance machine learning tasks. Throughout this paper, this configuration is referred to as **M4 Max**.
2) **Apple M3 Max MacBook Pro:** Operating on macOS Sequoia 15.1, this system uses the M3 Max chip with 96 GB of unified memory, designed for energy-efficient, high-performance workloads. Referred to as **M3 Max**.
3) **NVIDIA RTX A6000 Ada Generation GPU:** Equipped with 48 GB of GDDR6 ECC memory, 18,176 CUDA cores, and 568 Tensor cores. Tested on Windows 11 Pro (**RTX A6000 [Windows]**) and within the Windows Subsystem for Linux (WSL) (**RTX A6000**).
4) **NVIDIA GeForce RTX 4090 GPU:** Running on Ubuntu 22.04 via WSL, this GPU offers 24 GB of GDDR6X memory, 16,384 CUDA cores, and 512 Tensor cores. Referred to as **RTX 4090**.
5) **NVIDIA GeForce RTX 4090 Laptop GPU:** This high-performance mobile GPU features 16 GB of GDDR6 memory, 9,728 CUDA cores, and 304 Tensor cores. Tested on Windows 11 Pro (**RTX 4090 Laptop [Windows]**) and WSL (**RTX 4090 Laptop**).
6) **NVIDIA Jetson AGX Orin Developer Kit:** Running Ubuntu 20.04, this system combines a 12-core Arm Cortex-A78AE CPU with an Ampere GPU (2,048 CUDA cores and 64 Tensor cores) and 64 GB of memory, delivering up to 275 TOPS for edge AI applications. Referred to as **Jetson AGX Orin**.

The testing environment leverages Ollama 0.5.4 for executing LLMs on local GPUs, seamlessly integrated with a custom Python evaluation framework. This setup enables efficient utilization of computational resources, streamlining evaluation processes while maintaining adaptability across diverse computing environments.

IV. EXPERIMENTAL RESULTS AND DISCUSSIONS

A. Improved Few-Shot Learning with GMRID v3

We evaluated four language models on the updated *GMRID v3* dataset: Llama-3.1-8B, Llama-3.1-70B, GPT-4o-mini, and GPT-4o, previously analyzed in (10). GPT-4o-mini and GPT-4o were evaluated via OpenAI API, while Llama-3.1 models were tested on Nvidia H100 GPUs using Hugging Face Transformers (12) with 4-bit quantization through bitsandbytes[3].

Table II compares model performance on *GMRID v2* and *GMRID v3*. The results show consistent improvements with *GMRID v3*: Llama-3.1-8B's peak F1 score increased from 87.08% to 89.50%, while Llama-3.1-70B improved from 94.01% to 95.51%. These gains demonstrate the effectiveness

[3]https://github.com/bitsandbytes-foundation/bitsandbytes

of our refined dataset and prompting approach, establishing baseline performance metrics for future Ollama experiments.

Model	Dataset	0-shot F1 (%)	Best-shot F1 (%)
Llama-3.1-8B	GMRID_v2	71.51	87.08 (1-shot)
	GMRID_v3	74.61	89.50 (1-shot)
Llama-3.1-70B	GMRID_v2	92.54	94.01 (10-shot)
	GMRID_v3	94.05	95.51 (10-shot)
gpt-4o-mini	GMRID_v2	92.18	93.86 (1-shot)
	GMRID_v3	93.68	94.92 (10-shot)
gpt-4o	GMRID_v2	94.97	98.76 (5-shot)
	GMRID_v3	95.39	99.74 (4-shot)

TABLE II
F1 SCORE COMPARISON OF ZERO-SHOT AND BEST FEW-SHOT RESULTS FOR *GMRID v2* AND *GMRID v3* DATASETS.

B. Performance Trade-offs Between Quantized and FP16 Precision LLMs

Ollama models default to `Q4_K_M` quantization, a technique that significantly reduces memory usage and computational overhead. This raises a critical question: how do these quantized models perform compared to their FP16 precision counterparts? To investigate, we evaluated both 4-bit and FP16 variants of models with up to 14 billion parameters on two hardware platforms—the RTX 4090 and M4 Max.

Fig. 1 illustrates the comparative analysis of performance and throughput across these platforms. Specifically, we examined models ranging in size from 0.5 billion to 14 billion parameters, capturing metrics such as F1 score and throughput.

Key findings include:

- Models in 4-bit precision consistently demonstrate faster evaluation times compared to FP16 across both platforms, making them highly efficient for inference tasks.
- FP16 models occasionally achieve marginally higher F1 scores—such as with smaller models like *qwen2.5:1.5b*, which showed a 7.34% improvement on the RTX 4090. However, for most models, these differences are often negligible or even negative.
- 4-bit quantization delivers significant throughput gains on both platforms, reinforcing its suitability for resource-constrained edge AI deployments.
- For larger models, such as *llama3.2-vision:11b* and *qwen2.5:14b*, FP16 exhibited substantial throughput reductions on both platforms, highlighting the computational efficiency of 4-bit quantization for high-demand tasks.

An important consideration in the trade-offs between 4-bit quantized and FP16 models lies in their memory efficiency. Table III illustrates the significant differences in memory usage between these two precision types:

- 4-bit quantized models consistently use 30.0–64.5% less memory during loading compared to FP16. For instance, *qwen2.5:1.5b* requires only 2.1GB in 4-bit versus 4.3GB in FP16. This disparity grows with model size: *qwen2.5:14b* needs 11GB in 4-bit but 31GB in FP16.
- During evaluation, 4-bit models reduce memory usage by 20.0–52.6%. For example, the *qwen2.5:3b* model uses 4.9GB in 4-bit versus 9.3GB in FP16—a nearly 90% increase. Interestingly, this is higher than the 9.0GB required for the evaluation of the larger *qwen2.5:7b* model in 4-bit. Moreover, despite its higher memory demand, the FP16 *qwen2.5:3b* performs worse, achieving a lower F1 score (90.30% vs. 92.60%) and reduced throughput (6,584 vs. 6,607 tokens/s).
- The reduced memory footprint of 4-bit models facilitates deployment on hardware with limited VRAM, such as consumer GPUs or edge devices. For instance, the *qwen2.5:7b* model requires 16GB/19GB (Load/Eval) in FP16 but only 6GB/9GB in its quantized form. This makes 4-bit models practical for devices with modest memory capacities while preserving competitive performance metrics.

Model	Memory (Load/Eval)	% Mem. Reduction (Load/Eval)	F1 (%)	T-put (t/s)
`qwen2.5:0.5b`	1.4GB/2.4GB	30.0%/20.0%	48.20	13,123
`qwen2.5:0.5b (FP16)`	2.0GB/3.0GB	—	46.70	12,428
`llama3.2:1b`	2.8GB/5.3GB	30.0%/18.5%	63.50	5,035
`llama3.2:1b (FP16)`	4.0GB/6.5GB	—	63.50	4,084
`qwen2.5:1.5b`	2.1GB/3.4GB	51.2%/39.3%	73.70	7,419
`qwen2.5:1.5b (FP16)`	4.3GB/5.6GB	—	81.00	5,611
`llama3.2:3b`	4.0GB/8.2GB	53.5%/31.7%	83.60	9,766
`llama3.2:3b (FP16)`	8.6GB/12GB	—	82.20	5,654
`qwen2.5:3b`	3.1GB/4.9GB	58.7%/47.3%	91.30	8,653
`qwen2.5:3b (FP16)`	7.5GB/9.3GB	—	90.30	6,584
`qwen2.5:7b`	6.0GB/9.0GB	62.5%/52.6%	92.60	6,607
`qwen2.5:7b (FP16)`	16GB/19GB	—	93.00	2,491
`llama3.1:8b`	6.7GB/11GB	60.6%/50.0%	92.00	6,761
`llama3.1:8b (FP16)`	17GB/22GB	—	94.10	3,805
`llama3.2-vision:11b`	12GB/13GB	52.0%/50.0%	92.50	6,293
`llama3.2-vision:11b (FP16)`	25GB/26GB	—	94.40	468
`qwen2.5:14b`	11GB/18GB	64.5%/52.6%	94.90	3,995
`qwen2.5:14b (FP16)`	31GB/38GB	—	94.20	89
`qwen2.5:32b`	23GB/31GB	N/A	96.90	2,128
`llama3.1:70b`	44GB/55GB	N/A	95.50	94
`llama3.3:70b`	46GB/58GB	N/A	95.70	73
`qwen2.5:72b`	51GB/63GB	N/A	96.90	109
`llama3.2-vision:90b`	60GB/63GB	N/A	96.10	56

TABLE III
PERFORMANCE AND RESOURCE REQUIREMENTS OF LLMS. `FP16` REFERS TO THE FP16 PRECISION VARIANT OF EACH MODEL. "% MEM. REDUCTION (LOAD/EVAL)" SHOWS MEMORY SAVINGS FOR LOADING AND EVALUATION WITH Q4_K_M QUANTIZATION COMPARED TO FP16. F1 SCORES ARE PERCENTAGES, AND THROUGHPUT (T-PUT) IS MEASURED IN TOKENS PER SECOND (T/S). METRICS WERE COLLECTED ON THE RTX 4090 PLATFORM.

C. Performance Analysis Across All Quantized Models

Due to hardware limitations, only the RTX A6000 [Windows], RTX A6000, RTX 4090, and M3 Max systems successfully completed the evaluation of all quantized models within a reasonable timeframe. Fig. 2 presents the performance patterns across model sizes on these platforms. As the RTX A6000 [Windows] and RTX A6000 configurations produce nearly identical F1 scores across all models and shot configurations, the RTX A6000 [Windows] is excluded from this analysis.

Our analysis reveals several key findings:

a) Small Models (0.5b–3b): These models exhibit limited scalability with the number of shots:

- Models smaller than 1.5b, such as `qwen2.5:0.5b` and `llama3.2:1b`, show modest F1 score improvements or

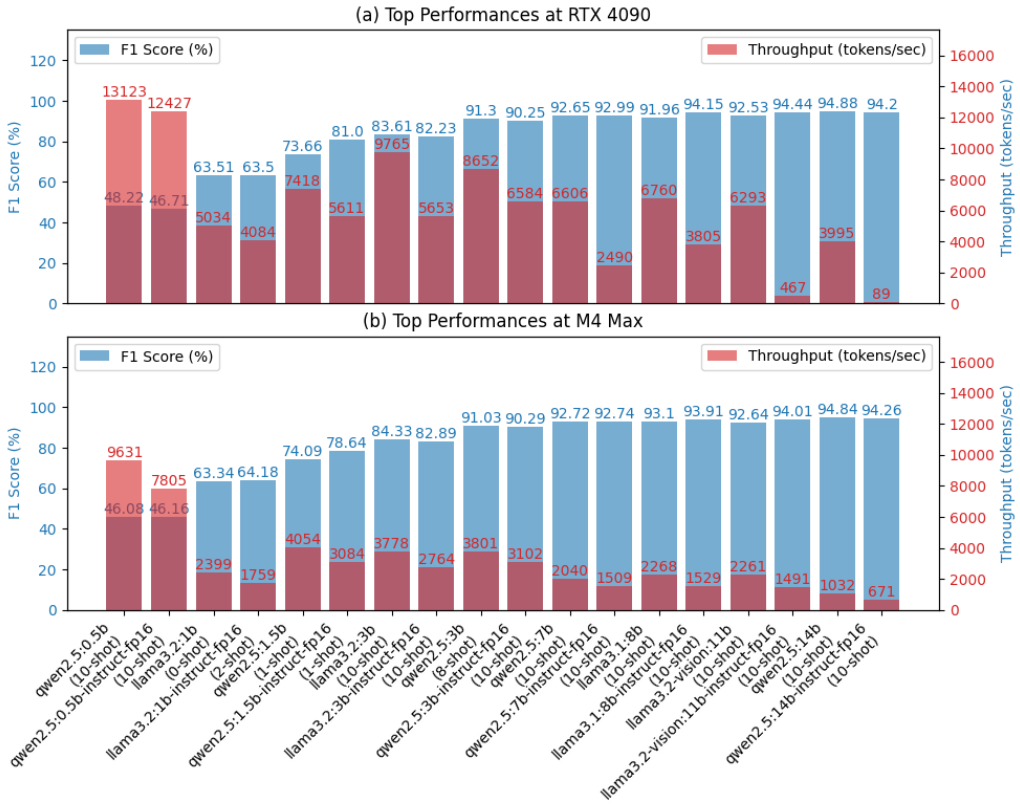

Fig. 1. Performance Comparison Between 4-bit Quantized Models and FP16 Models Across Hardware Platforms. Subplot (a) displays the performance of various models on the RTX 4090 platform, while subplot (b) presents the corresponding performance on the M4 Max platform.

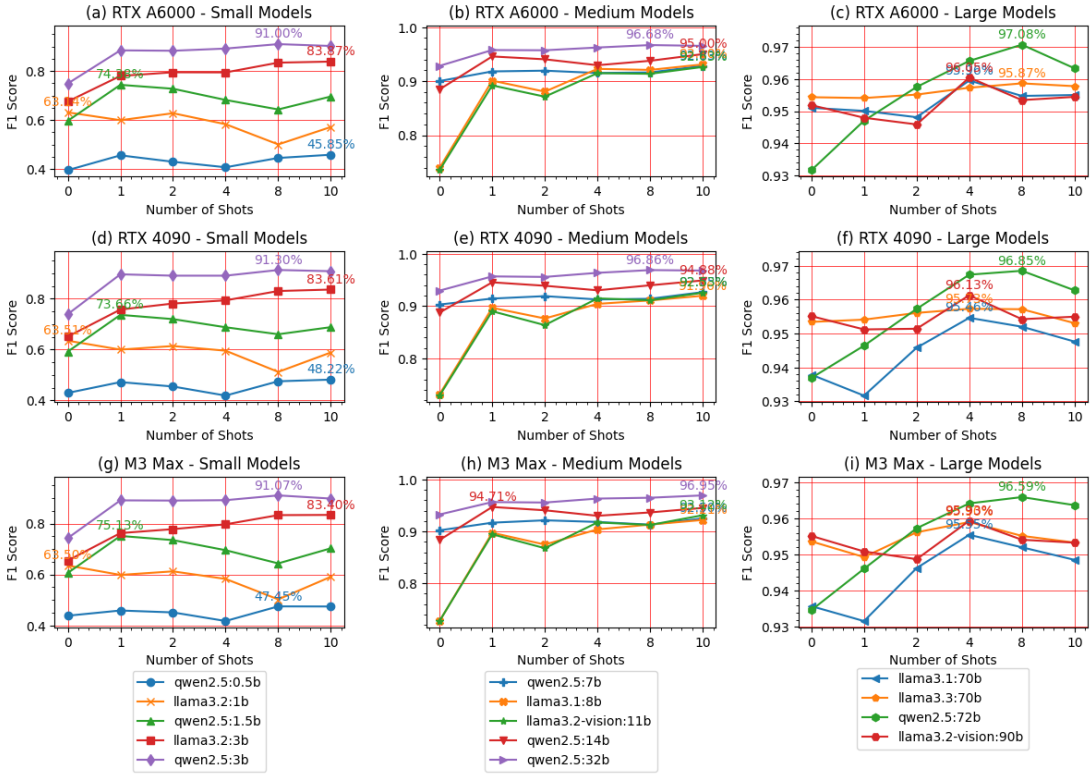

Fig. 2. F1 scores of quantized models on RTX A6000, RTX 4090, and M3 Max across different model sizes and shot counts.

even slight decreases across 0–10 shots, highlighting their limited few-shot learning capabilities.

- `qwen2.5:1.5b` achieves a notable F1 score of 73.66–75.13% at 1 shot but exhibits diminishing returns with additional shots.
- Performance patterns remain consistent across all three GPU platforms.
- `qwen2.5:3b` achieves the highest performance among small models, reaching an F1 score of 91.00–91.30% at 8 shots.

b) Medium Models (7b–14b): These configurations demonstrate robust scaling:

- Significant F1 score improvements are observed with few-shot learning, particularly in the 0–1 shot range.
- Performance remains consistent across the platforms, underscoring strong quantization robustness.
- `qwen2.5:32b` leads this category, achieving a remarkable F1 score of 96.68–96.95% at 8 or 10 shots.

c) Large Models (70b–90b): These models showcase optimal performance:

- Exceptional F1 scores exceeding 95% are achieved with as few as 4 shots.
- Minimal variation across GPU platforms indicates effective quantization scaling for larger models.
- Peak performance is achieved by `qwen2.5:72b`, reaching an impressive F1 score of 96.59–97.08% at 8 shots.

This analysis reveals that quantization effectively preserves model performance while enabling deployment across diverse GPU architectures. Larger models consistently outperform smaller variants, albeit at the cost of increased computational requirements. The results also highlight the effectiveness of few-shot learning in quantized settings, with most models demonstrating significant improvements within first four shots.

D. Cross-Platform Performance Analysis and Hardware-Specific Insights

Tables IV, V, and VI present performance metrics under few-shot settings yielding the highest F1 scores, grouped by model size. Metrics include F1 scores, throughput (tokens/second), and mean evaluation time (seconds). Large models (70B–90B) are evaluated only on high-capacity systems (RTX A6000, RTX 4090, and M3 Max) due to memory constraints, while smaller models are tested across all platforms.

Key findings include:

1) Ollama's `Q4_K_M` quantization outperforms bitsandbytes' 4-bit implementation. For instance, `llama3.1:8b` achieves a mean F1 score of 92.78% (±0.50%), surpassing its baseline of 89.50%, while `llama3.1:70b` improves to 95.83% (±0.07%).
2) On identical GPUs, WSL usually achieves higher throughput and lower evaluation times than Windows, particularly for smaller models. For example, on the RTX A6000, `qwen2.5:0.5b` processes 16,933 tokens/s on WSL compared to 816 tokens/s on Windows (21× faster). This gap narrows for larger models, with differences as low as 15% at 90B.
3) Larger models predictably reduce throughput and increase evaluation times. For example, on the RTX 4090, throughput drops from 13,124 tokens/s (`qwen2.5:0.5b`) to 56 tokens/s (`llama3.2-vision:90b`), while evaluation time rises from 0.16 seconds to 26.505 seconds.
4) The `qwen2.5` series consistently delivers superior F1 scores across all platforms. Notably, `qwen2.5:32b` achieves an F1 score of 96.75% (±0.11%) with practical inference speeds, demonstrating balanced optimization of performance and efficiency.

Model	Platform/Stats	F1 (%)	T-put (t/s)	Time (s)
qwen2.5:0.5b	RTX A6000	45.85	16933	0.124
	RTX A6000 [Windows]	45.85	816	2.573
	RTX 4090	48.22	13124	0.160
	M3 Max	47.45	7178	0.258
	M4 Max	46.08	9632	0.218
	RTX 4090 Laptop	46.58	6907	0.304
	RTX 4090 Laptop [Windows]	46.58	776	2.707
	Jetson AGX Orin	46.65	1762	1.192
	mean	46.66	7141	0.942
	std	0.77	5561	1.032
llama3.2:1b	RTX A6000	63.14	3922	0.276
	RTX A6000 [Windows]	63.14	401	2.698
	RTX 4090	63.51	5035	0.215
	M3 Max	63.50	1899	0.570
	M4 Max	63.34	2400	0.451
	RTX 4090 Laptop	62.65	1893	0.572
	RTX 4090 Laptop [Windows]	62.65	361	2.998
	Jetson AGX Orin	63.41	352	3.074
	mean	63.17	2033	1.357
	std	0.33	1622	1.223
qwen2.5:1.5b	RTX A6000	74.38	8744	0.134
	RTX A6000 [Windows]	74.38	456	2.571
	RTX 4090	73.66	7419	0.158
	M3 Max	75.13	2224	0.527
	M4 Max	74.09	4055	0.289
	RTX 4090 Laptop	74.94	3768	0.311
	RTX 4090 Laptop [Windows]	74.94	429	2.733
	Jetson AGX Orin	72.16	740	1.583
	mean	74.21	3479	1.038
	std	0.90	2983	1.028
llama3.2:3b	RTX A6000	83.87	12220	0.171
	RTX A6000 [Windows]	83.87	803	2.601
	RTX 4090	83.61	9766	0.214
	M3 Max	83.40	2521	0.829
	M4 Max	84.33	3779	0.553
	RTX 4090 Laptop	84.18	4850	0.380
	RTX 4090 Laptop [Windows]	84.18	662	2.783
	Jetson AGX Orin	84.34	596	3.091
	mean	83.97	4400	1.328
	std	0.32	4115	1.182
qwen2.5:3b	RTX A6000	91.00	10118	0.183
	RTX A6000 [Windows]	91.00	710	2.608
	RTX 4090	91.30	8653	0.214
	M3 Max	91.07	1197	1.547
	M4 Max	91.03	3802	0.487
	RTX 4090 Laptop	90.65	4797	0.386
	RTX 4090 Laptop [Windows]	90.65	659	2.808
	Jetson AGX Orin	90.46	730	2.536
	mean	90.89	3833	1.346
	std	0.26	3537	1.089

TABLE IV
PERFORMANCE METRICS FOR SMALL MODELS (0.5B-3B). F1 SCORES ARE EXPRESSED AS PERCENTAGES, THROUGHPUT (T-PUT) IS MEASURED IN TOKENS PER SECOND (T/S), AND MEAN EVALUATION TIME (TIME) IS MEASURED IN SECONDS (S).

E. Energy Efficiency in LLMs on RTX A6000

Energy efficiency evaluates the trade-off between model performance and energy consumption. This subsection analyzes energy efficiency metrics for models running on RTX A6000, as shown in Fig. 3.

Fig. 3(a) compare *F1 Score* with *Energy per Inference*:

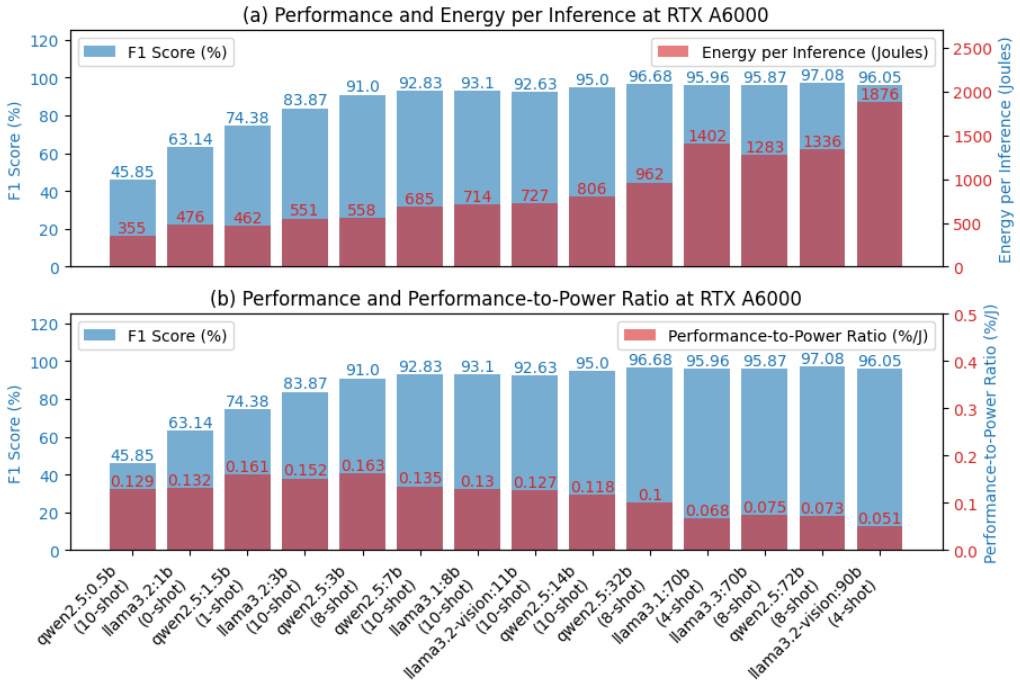

Fig. 3. Performance and Energy Efficiency Metrics on RTX A6000. (a) F1 Score (%) vs. Energy per Inference (Joules). (b) F1 Score (%) vs. Performance-to-Power Ratio (%/J).

- Large models such as `qwen2.5:72b` and `llama3.2-vision:90b` achieve high F1 scores (97.08% and 96.05%) but consume 1,283–1,876 Joules per inference.
- Small models like `llama3.2-3b` and `qwen2.5:3b` consume significantly less energy (551 and 558 Joules) while achieving F1 scores of 83.87% and 91.0%, making them suitable for resource-constrained scenarios.

Fig. 3(b) highlights *Performance-to-Power Ratio (%/J)*:

- Small models like `qwen2.5:1.5b` and `qwen2.5:3b` achieve the highest ratios (0.161 and 0.163 %/J), ideal for energy-efficient deployments.
- Large models such as `llama3.2-vision:90b`, while achieving an F1 score of 96.05%, have lower energy efficiency (0.051 %/J), reflecting diminishing returns in energy usage at larger scales.

This analysis highlights the need to align model selection with deployment priorities. Large models like `qwen2.5:72b` excel in raw performance, while smaller models (`qwen2.5:1.5b` or `qwen2.5:3b`) balance performance and energy efficiency, supporting informed decision-making for diverse use cases.

V. Conclusion

This study evaluates LLMs across diverse hardware platforms using the Ollama framework, offering insights into efficient and sustainable edge AI deployments. Ollama's `Q4_K_M` quantization outperforms bitsandbytes, with `llama3.1:8b` and `llama3.1:70b` achieving 3.28% and 0.32% higher F1 scores, respectively. The framework's cross-platform stability further underscores its suitability for edge environments.

Our results reveal a trade-off between model size, energy consumption, and performance. Larger models show increased cross-platform stability, with F1 score variances narrowing from 0.26–0.9% (small models) to 0.07–0.22% (large models), but at the cost of higher energy usage—up to 1,876 Joules per inference. Medium-sized models (7B–32B) offer a strong balance, with `qwen2.5:32B` achieving a 96.75% ($\pm$0.11%) F1 score at practical speeds and moderate energy consumption.

We also find that Windows Subsystem for Linux (WSL) can deliver up to 21× higher throughput for smaller models compared to native Windows, enabling efficient deployment on consumer-grade GPUs. These results show that, with the right configurations, energy-efficient edge AI is both feasible and accessible. By reducing infrastructure needs and supporting local execution, edge deployments can mitigate the environmental footprint of LLMs and promote more sustainable AI systems.

Future work may include:

1) Validating observed performance and efficiency trends across more datasets and domains.
2) Benchmarking Ollama's `Q4_K_M` quantization against alternatives (e.g., INT8).
3) Including broader sustainability metrics such as hardware lifecycle and total cost of ownership.
4) Exploring practical deployment and scalability challenges through real-world case studies.

This work advances the democratization of AI, promoting

Model	Platform/Stats	F1 (%)	T-put (t/s)	Time (s)
qwen2.5:7b	RTX A6000	92.74	7298	0.288
	RTX A6000 [Windows]	92.83	784	2.681
	RTX 4090	92.65	6607	0.318
	M3 Max	92.50	1194	1.760
	M4 Max	92.72	2041	1.030
	RTX 4090 Laptop	92.46	3944	0.533
	RTX 4090 Laptop [Windows]	92.46	705	2.983
	Jetson AGX Orin	92.78	410	5.130
	mean	92.64	2873	1.840
	std	0.14	2582	1.579
llama3.1:8b	RTX A6000	93.10	7855	0.266
	RTX A6000 [Windows]	93.10	778	2.684
	RTX 4090	91.96	6761	0.309
	M3 Max	92.21	1430	1.461
	M4 Max	93.10	2269	0.921
	RTX 4090 Laptop	93.19	3935	0.531
	RTX 4090 Laptop [Windows]	93.19	706	2.959
	Jetson AGX Orin	93.10	406	5.146
	mean	92.87	3017	1.785
	std	0.46	2704	1.597
llama3.2-vision:11b	RTX A6000	92.63	7944	0.263
	RTX A6000 [Windows]	92.63	780	2.680
	RTX 4090	92.53	6293	0.332
	M3 Max	93.12	1232	1.696
	M4 Max	92.64	2261	0.924
	RTX 4090 Laptop	93.39	3927	0.532
	RTX 4090 Laptop [Windows]	93.39	707	2.956
	Jetson AGX Orin	92.82	456	4.585
	mean	92.89	2950	1.746
	std	0.33	2657	1.447
qwen2.5:14b	RTX A6000	95.18	4105	0.512
	RTX A6000 [Windows]	95.00	728	2.888
	RTX 4090	94.88	3995	0.526
	M3 Max	94.78	343	3.418
	M4 Max	94.84	1033	2.035
	RTX 4090 Laptop	95.17	1988	1.057
	RTX 4090 Laptop [Windows]	95.09	638	3.292
	Jetson AGX Orin	95.06	204	10.304
	mean	95.00	1629	3.004
	std	0.14	1487	2.971
	mean	94.99	1591	6.191
	std	0.16	1523	9.086
qwen2.5:32b	RTX A6000	96.68	2048	0.906
	RTX A6000 [Windows]	96.68	560	3.315
	RTX 4090	96.86	2128	0.872
	M3 Max	96.95	259	8.131
	M4 Max	96.86	143	12.995
	RTX 4090 Laptop	96.68	221	8.404
	RTX 4090 Laptop [Windows]	96.60	228	8.130
	Jetson AGX Orin	96.68	95	19.497
	mean	96.75	710	7.781
	std	0.11	806	5.916

TABLE V
PERFORMANCE METRICS FOR MEDIUM MODELS (7B-32B). F1 SCORES ARE EXPRESSED AS PERCENTAGES, THROUGHPUT (T-PUT) IS MEASURED IN TOKENS PER SECOND (T/S), AND MEAN EVALUATION TIME (TIME) IS MEASURED IN SECONDS (S).

Model	Platform/Stats	F1 (%)	T-put (t/s)	Time (s)
llama3.1:70b	RTX A6000	95.96	621	2.374
	RTX A6000 [Windows]	95.96	312	4.724
	RTX 4090	95.46	94	15.657
	M3 Max	95.55	120	12.293
	mean	95.73	287	8.762
	std	0.23	211	5.411
llama3.3:70b	RTX A6000	95.87	937	1.966
	RTX A6000 [Windows]	95.87	426	4.322
	RTX 4090	95.73	73	20.241
	M3 Max	95.90	37	39.523
	mean	95.84	368	16.513
	std	0.07	362	15.030
qwen2.5:72b	RTX A6000	97.08	515	3.604
	RTX A6000 [Windows]	97.08	314	5.911
	RTX 4090	96.85	109	17.072
	M3 Max	96.59	116	16.055
	mean	96.90	263	10.660
	std	0.20	167	5.970
llama3.2-vision:90b	RTX A6000	96.05	117	12.611
	RTX A6000 [Windows]	96.05	102	14.460
	RTX 4090	96.13	56	26.505
	M3 Max	95.93	85	17.379
	mean	96.04	90	17.739
	std	0.07	23	5.339

TABLE VI
PERFORMANCE METRICS FOR LARGE MODELS (70B–90B). F1 SCORES ARE EXPRESSED AS PERCENTAGES, THROUGHPUT (T-PUT) IS MEASURED IN TOKENS PER SECOND (T/S), AND MEAN EVALUATION TIME (TIME) IS MEASURED IN SECONDS (S).

inclusive, sustainable, and privacy-preserving deployments that reduce environmental impact, empower underserved communities, and bridge digital divides.

REFERENCES

[1] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, and J. Chen, "A review on edge large language models: Design, execution, and applications," *J. ACM*, vol. 37, no. 4, p. 111, 2024.

[2] N. Dhar, B. Deng, D. Lo, X. Wu, L. Zhao, and K. Suo, "An empirical analysis and resource footprint study of deploying large language models on edge devices," in *ACMSE*, 2024.

[3] A. Singh, N. P. Patel, A. Ehtesham, S. Kumar, and T. T. Khoei, "A survey of sustainability in large language models: Applications, economics, and challenges," *arXiv preprint arXiv:2412.04782*, 2024.

[4] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and H.-K. Song, "Shortened llama: A simple depth pruning for large language models," *ICLR*, 2024.

[5] E. Frantar and D. Alistarh, "Optimal brain compression: A framework for accurate post-training quantization and pruning," in *NeurIPS*, 2022.

[6] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2. 5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[7] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[8] K. T. Chitty-Venkata, S. Raskar, B. Kale, F. Ferdaus, A. Tanikanti, K. Raffenetti, V. Taylor, M. Emani, and V. Vishwanath, "Llm-inference-bench: Inference benchmarking of large language models on ai accelerators," *arXiv preprint arXiv:2411.00136*, 2024.

[9] V. Pandelea, E. Ragusa, P. Gastaldo, and E. Cambria, "Selecting language models features via software-hardware co-design," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[10] D. Huang, X. Fu, X. Yin, H. Pen, and Z. Wang, "Automating maritime risk data collection and identification leveraging large language models," in *2024 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2024, pp. 433–439.

[11] S. Hisaharo, Y. Nishimura, and A. Takahashi, "Optimizing llm inference clusters for enhanced performance and energy efficiency," *Authorea Preprints*, 2024.

[12] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.